

Informe Resultados de Tratamiento de Imagen

Jhoan Ricardo Ávila Gutiérrez

Esp. Inteligencia Artificial, Universidad de Cundinamarca

Clase: Percepción Computacional

Profesor: John Jairo Duran

Mayo 24, 2026

Resumen

Este informe tiene como finalidad el análisis y estudio del proceso de percepción computacional centrado en la detección y localización de objetos en múltiples imágenes simultáneamente.

El código empleado para este informe fue diseñado en las sesiones sincrónicas y personalizado para ampliar su alcance, pensado para ejecutarse tanto en entorno de la nube (Google Colab) como en entornos locales. Emplea una red neuronal convolucional YOLOv8 (You Only Look Once) para identificar entidades visuales. Los resultados de la detección son filtrados mediante un umbral de confianza (> 0.40) y mapeados en coordenadas en tiempo real.

Finalmente, la información obtenida se consolida de manera automática en un reporte en formato CSV.

Palabras clave: Percepción Computacional, YOLOv8, Google Colab, Detección de Objetos, Procesamiento de Imágenes.

Introducción

La detección de objetos en imágenes digitales constituye uno de los pilares fundamentales de la Percepción Computacional moderna. A diferencia de la clasificación de imágenes tradicional, la detección de objetos aborda un problema de mayor complejidad: identificar qué objetos están presentes, cuántos hay y dónde se localizan espacialmente mediante coordenadas de encuadre o cajas delimitadoras (Redmon, 2016).

La introducción de arquitecturas de una sola etapa como YOLO (You Only Look Once) revolucionó la predicción de límites y probabilidades de clase en una sola red, posibilitando tasas de procesamiento superiores a los 30 fotogramas por segundo (FPS).

Por esta razón, el objetivo de este informe es detallar la funcionalidad del código, desglosar su lógica por medio de los scripts empleados, analizar la arquitectura escogida de desarrollo y concluir que alcance se puede lograr con lo realizado.

Nota: [Repositorio del código para detección de objetos.](#)

Análisis y finalidad del código

El propósito de este código es permitir al usuario procesar lotes de imágenes (cargados manualmente), extraer los objetos presentes en la imagen, imprimir en pantalla la detección y descargar esta información en un formato compatible (CSV).

Para entender cómo trabaja, el código se ha estructurado en bloques de scripts modulares con funcionalidades bien definidas. A continuación, se detalla explicativamente cada uno de ellos:

Módulo A: Llamado de dependencias y detección de entorno

- *Llamado de librerías*: Si el script se ejecuta en un entorno sin las librerías requeridas la excepción invoca el gestor de paquetes de Python para instalar *ultralytics* y *opencv-python*. Esto garantiza la ejecución correcta de las funcionalidades.
- *Detección de entorno*: Evalúa si la ejecución de código se realizara desde Google Colab. Si no detecta Colab, el script asume una ejecución en entorno local y utiliza el sistema de visualización gráfica estándar de OpenCV.

```
import sys

try:
    from ultralytics import YOLO
    import cv2
except:
    print("Instalando dependencias...")
    !pip install -q ultralytics opencv-python
    from ultralytics import YOLO
    import cv2

import os
import csv
from datetime import datetime

# Detectar si estamos en Colab
EN_COLAB = "google.colab" in sys.modules

if EN_COLAB:
    from google.colab import files
    from google.colab.patches import cv2_imshow
```

Módulo B: Gestión de directorios

Este bloque se encarga de crear el ecosistema de almacenamiento local necesario para cargar las imágenes que serán analizadas.

- *Definición de entradas y salidas:* Se configura el nombre de la carpeta donde se almacenará el lote de imágenes a procesar y se configura el nombre del archivo salida csv.

```
if EN_COLAB:
    print("Sube tus imágenes (pueden ser varias)...")
    uploaded = files.upload()

    # Crear carpeta
    os.makedirs("imagenes", exist_ok=True)

    # Guardar archivos en carpeta
    for nombre in uploaded.keys():
        with open(os.path.join("imagenes", nombre), "wb") as f:
            f.write(uploaded[nombre])
```

Modulo C: Carga y preparación del modelo YOLO

La clase YOLO del framework de *Ultralytics* encapsula el ciclo de vida del modelo de red profunda. Al invocarlo, el script busca localmente el archivo, si no se encuentra en el directorio actual la biblioteca lo descarga automáticamente desde los repositorios oficiales en GitHub.

- *Modelo YOLO:* Este modelo fue pre-entrenado en la gigantesca base de datos COCO, lo que le permite reconocer 80 clases comunes de objetos cotidianos con un desempeño asombroso y de manera inmediata, aplicando el concepto científico de Transfer Learning (aprendizaje por transferencia).

```
# 5. CARGAR MODELO
print("Cargando modelo YOLO...")
model = YOLO("yolov8n.pt")
print("Modelo listo")
```

Módulo D: Bucle de procesamiento, extracción de coordenadas y filtrado de predicción

Este fragmento es el motor del código. Es donde se procesa cada imagen y se extrae de manera estructurada los atributos de los objetos detectados.

- *Inferencia del modelo:* Envía la imagen a través de las capas convolucionales de la red neuronal y como resultado YOLO evalúa la composición de píxeles y genera un mapeo espacial con propuestas de objetos.
- *Extracción de atributos desde Tensor:* Primero, el tensor realiza un mapeo de clases y lo traduce consultando el diccionario de metadatos del modelo (*model.names[cls_id]*). Segundo, mapea probabilísticamente de que la predicción del objeto sea correcta. Y tercero, extrae las coordenadas en píxeles.
- *Filtro de clasificación:* Este es el núcleo del código. El umbral de confianza del 40% actúa como un filtro de ruido. Si el modelo detecta la silueta de un objeto, pero este tiene menos del 40% de certeza lo omite. Esto previene los falsos positivos.
- *Personalización del código:* El código trabajado en las sesiones sincrónicas se limitaba a detección únicamente de personas, esta nueva actualización permite escanear la imagen más a profundidad para detectar si hay presencia o coincidencia de algún objeto asociado a la biblioteca de COCO.

Como mejora visual, se redondea las coordenadas a 2 decimales y el nivel de confianza a 4 decimales para estandarizar los datos. Además, se agrega una marca de tiempo con la hora exacta en la que se completó el análisis de dicho objeto específico.

```

for archivo in os.listdir(CARPETA_IMAGENES):
    ruta = os.path.join(CARPETA_IMAGENES, archivo)

    # Filtrar solo imágenes
    if not archivo.lower().endswith((".jpg", ".jpeg", ".png")):
        continue

    print(f"Procesando: {archivo}")

    results = model(ruta)

    for r in results:
        if r.bboxes is None:
            continue

        for box in r.bboxes:
            # 1. Obtener el ID de la clase y su nombre en texto
            cls_id = int(box.cls[0].item())
            nombre_clase = model.names[cls_id]

            # 2. Obtener el nivel de confianza
            conf = float(box.conf[0].item())

            # 3. Obtener coordenadas del recuadro (x1, y1, x2, y2)
            x1, y1, x2, y2 = box.xyxy[0].tolist()

```

```

# Filtramos por confianza
if conf > 0.4:
    # Guardamos todos los datos relevantes del objeto actual
    resultados.append([
        archivo,
        nombre_clase,                # Ej: 'person', 'car', 'dog'
        round(conf, 4),              # Confianza redondeada a 4 decimales
        round(x1, 2),                # Coordenada X1
        round(y1, 2),                # Coordenada Y1
        round(x2, 2),                # Coordenada X2
        round(y2, 2),                # Coordenada Y2
        datetime.now().strftime("%Y-%m-%d %H:%M:%S") # Fecha y hora limpia
    ])

```

Módulo E: Visualización gráfica

Este módulo se encarga de mostrar al usuario las predicciones del modelo con una retroalimentación visual.

- *Metodo r.plot()*: Dibuja dinámicamente un lienzo sobre la imagen original en formato de matriz OpenCV (BGR), trazando rectángulos de colores diferenciados para cada objeto junto a su puntaje de confianza.

```
# Visualizar resultado
for r in results:
    img_resultado = r.plot()

    if EN_COLAB:
        cv2.imshow(img_resultado)
    else:
        cv2.imshow("Resultado", img_resultado)
        cv2.waitKey(500)
```

Módulo F: Salida de datos

- *Generación de reporte:* Se crea un archivo (*resultados_detallados.csv*) en la carpeta de salida y descarga.
- *Interpretación de datos:* Por cada imagen cargada en el lote se crea una fila con 5 columnas que registran los datos detectados por el modelo (nombre de la imagen, tipo de objeto detecto, porcentaje de confianza, coordenadas en la imagen donde se detectó y la fecha de procesamiento).

```
# 7. GUARDAR CSV
with open(ARCHIVO_SALIDA, mode='w', newline='') as f:
    writer = csv.writer(f)
    # Actualizamos los encabezados del CSV
    writer.writerow([
        "Imagen",
        "Objeto_Detectado",
        "Nivel_Confianza",
        "Coord_X1",
        "Coord_Y1",
        "Coord_X2",
        "Coord_Y2",
        "Fecha_Procesamiento"
    ])
    writer.writerows(resultados)

print("=====")
print("PROCESO FINALIZADO")
print("Archivo generado:", ARCHIVO_SALIDA)

# 8. DESCARGAR RESULTADOS (Colab)
if EN_COLAB:
    files.download(ARCHIVO_SALIDA)
```

Arquitectura del código

A diferencia de la clasificación de imágenes simple, la detección de objetos requiere resolver dos problemas:

- *Clasificación*: Identificar las clases o categorías presentes (persona, auto, perro).
- *Localización*: Dibujar una caja delimitadora que precise la posición exacta y la escala de cada objeto detectado dentro de la imagen.

Por esta razón para la ejecución de este código se optó por emplear las siguientes librerías:

- *Ultralytics YOLOv8*: La biblioteca ultralytics proporciona la API oficial para interactuar con los modelos YOLOv8. Estos modelos requieren una cantidad mínima de memoria RAM, lo que le permite cargarse rápidamente. Cuenta con 80 categorías comunes de clasificación de objetos sin necesidad de realizar un entrenamiento adicional (Jocher, 2023).
- *OpenCV*: Es el estándar para tareas de manipulación y análisis de imágenes. Permite visualizar en tiempo real el progreso de las detecciones mediante *r.plot()*, que dibuja los rectángulos de contorno, etiquetas y porcentajes de confianza calculados por YOLO.

Conclusiones

El desarrollo de este código representa un prototipo funcional altamente efectivo y bien diseñado para tareas de percepción computacional aplicada. La integración del modelo YOLOv8 con OpenCV y el manejo de excepciones en Google Colab permite un diseño funcional que prioriza la predicción del modelo y la experiencia del usuario.

Mediante el uso de librerías de aprendizaje profundo, se ha logrado reducir una inmensa complejidad matemática que conlleva las redes convolucionales profundas a una simplificada lógica de 160 líneas de código. Permitiendo no solo "dibujar cuadros" sobre los objetos detectados en las imágenes, sino que extrae y consolida esa información en un archivo CSV estructurado. Esto transforma píxeles no estructurados en información lista para ser procesada a gusto.

Referencias

- Redmon, J. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779-788.
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *YOLOv8 by Ultralytics (Version 8.0.0)* [Software]. Ultralytics, <https://github.com/ultralytics/ultralytics>. Recuperado de Github.